



SUBMITTED BY:

Name:	SAPURU NETESH
Roll no:	AV.SC.U4CSE25047
Branch:	CSE
Section:	CSE-A

SUBMITTED TO:

Name:	DR. Raj Kumar Batchu
Department:	School Of Computing
Designation:	Assitant.proffessor

Subject: Data Structures and Algorithms (DSA)

Credits: 05

Week-1

1)Aim: To implement recursion in Python by creating a function that prints numbers in descending order from a given number.

Program:

```
week1(a).py - C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week1(a).py (3.14.0)
File Edit Format Run Options Window Help
def launch(n):
    print(n)
    if n>1:
        n-=1
        return launch(n)
    print("LAUNCH")
result=launch(10)
```

Output:

```
>>>
===== RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week1(a).py =====
10
9
8
7
6
5
4
3
2
1
LAUNCH
>>>
```

Explanation:

- The function launch(n) prints the current value of n.
- If n is greater than 1, it decreases n by 1 and calls itself recursively.
- The recursion continues until n becomes 0, which is the base case, and the function stops.

Error Table:

S.No	Error Obtained	Rectification
01.	Didn't write the print statement after definition.	After execution I found the mistake and added it.

2)Aim: To implement a recursive binary search in Python to find the position (index) of a given element in a list. The program recursively checks each element of the list until the target element is found or the end of the list is reached.

Program:

```
week1(b).py - C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week1(b).py (3.14.0)
File Edit Format Run Options Window Help
def empid_Search(arr, target, n=0):
    if n==len(arr):
        return -1
    if arr[n]==target:
        return n
    return empid_Search(arr, target, n+1)
arr=[10,20,30,50,67,89,56]
result=empid_Search(arr,30)
if result!=-1:
    print("Element found at index:",result)
else:
    print("Element not found")
```

Output:

```
>>>
===== RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week1(b).py =====
Element found at index: 2
>>>
```

Explanation:

This program uses recursion to perform a linear search for an employee ID (or any target element) in a list.

1. The function `empid_Search(arr, target, n=0)` is defined with three parameters:
 - `arr`: The list in which the search is performed.
 - `target`: The element to be searched.
 - `n`: The current index, initialized to 0.
2. The statement `if n == len(arr): return -1` is the base condition. It checks whether all elements in the list have been examined. If the end of the list is reached without finding the target, the function returns -1.
3. The statement `if arr[n] == target: return n` compares the current element with the target. If they are equal, the function returns the current index where the target is found.
4. If the current element does not match the target, the function calls itself recursively with the next index using `return empid_search(arr, target, n + 1)`.
5. The list `arr = [10, 20, 30, 50, 607, 809, 506]` is created, and the function is called to search for the element 50.
6. The returned value is stored in the variable `result`.

- Error table:

3) Aim: This Python program uses recursion to calculate the value of p^n (p raised to the power n)

Program:

Output:

Explanation:

- The program is used to find the power of a number (p^n) using recursion.
- It takes two inputs from the user:
- $p \rightarrow$ Base number
- $n \rightarrow$ Exponent (power)
- The function `power(p, n)` is called to calculate the result.

- If the exponent n is **0**, the function returns **1** because any number raised to the power 0 is
- If n is not 0, the function calls itself with $n - 1$ and multiplies the result by p .
- This recursive process continues until the exponent becomes **0**.
- After reaching the base case, each recursive call returns its result, and the final value of p^n is obtained.
- The result is stored in the variable `result`.
- Finally, the program prints the calculated power of the given number.

Error Table:

S. No	Error Obtained	Rectification
01.	Not a int value of n we can also take float in place of int, I have taken float value to check.	By replacing int by float.

4)Aim: To write a Python program that generates and prints the Fibonacci series up to n terms using recursion.

Program:

```

week1(d).py - C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week1(d).py (3.14.0)
File Edit Format Run Options Window Help
def fib(n):
    if n<=1:
        return n
    return fib(n-1) + fib(n-2)
n=int (input ("Enter the value of n:"))
print ("The fibanocci series:")
for i in range (n):
    print(fib(i), end=" ")

```

Output:

```

>>>
===== RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week1(d).py =====
Enter the value of n:6
The fibanocci series:
0 1 1 2 3 5
>>>
===== RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week1(d).py =====
Enter the value of n:5
The fibanocci series:
0 1 1 2 3
>>>

```

Explanation:

- The program uses a recursive function `fib(n)` to generate Fibonacci numbers.
- The base condition returns 0 when $n = 0$ and 1 when $n = 1$.
- For values greater than 1, the function returns the sum of the previous two Fibonacci numbers: `fib(n-1) + fib(n-2)`.

- The user enters the number of terms (n) to be displayed.
- A for loop calls the fib () function repeatedly for each term from 0 to n-1.
- Finally, the program prints the Fibonacci series in sequence.

Error Table:

S. No	Error Obtained	Rectification
01.	No Errors obtained	-----

5)Aim: To write a Python program to find the factorial of a given number using recursion.

Program:

```

week1(e).py - C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week1(e).py (3.14.0)
File Edit Format Run Options Window Help
def fact(n):
    if n<=1:
        return n
    return n*fact(n-1)
n=int(input("enter the number to see the factorial of that number "))
print (fact (n))

```

Output:

```

>>>
===== RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week1(e).py =====
enter the number to see the factorial of that number 6
720
>>>
===== RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week1(e).py =====
enter the number to see the factorial of that number 7
5040
>>>

```

Explanation:

- The program defines a recursive function fact(n) to calculate the factorial of a number.
- The base condition checks if n <= 1. If true, it returns 1 (for factorial, 0! = 1 and 1! = 1).
- Otherwise, the function returns n * fact(n-1), multiplying the current number by the factorial of the previous number.
- The function is called with the value 6.
- Finally, the program prints the factorial of 6, which is 720.

Error Table:

S. No	Error Obtained	Rectification
01.	No Errors	-----

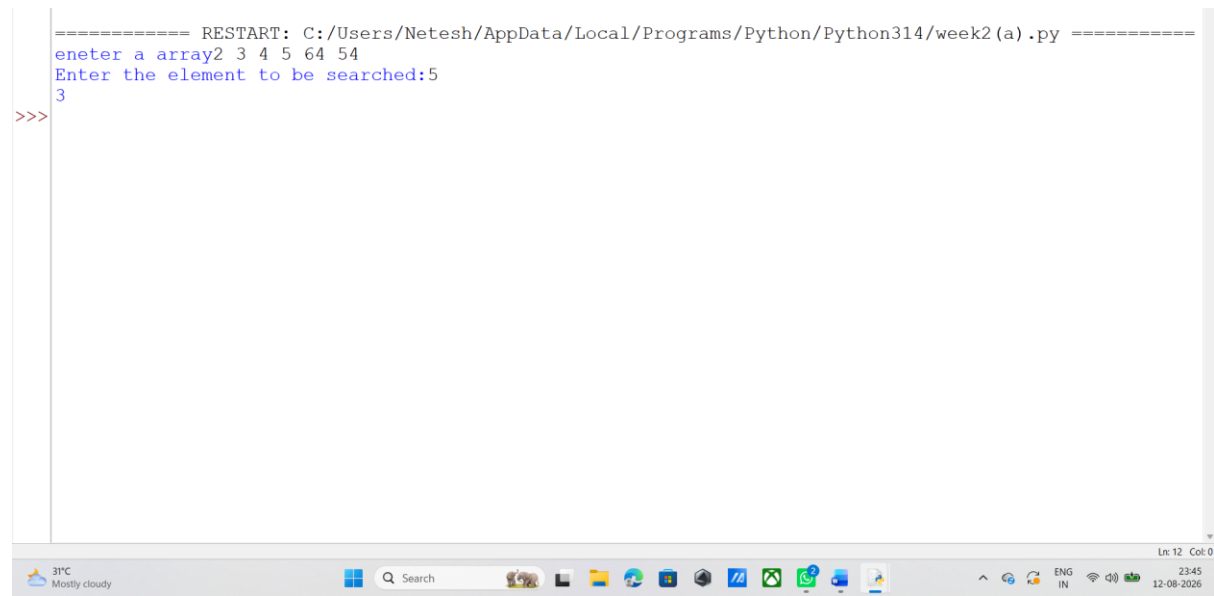
Week-2

1) Aim: To write a Python program to search for a given element in an array using the Linear Search technique and display its position.

Program:

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i]==key:  
            return i  
        else:  
            i=i+1  
    return 0  
arr=list(map(int,input("eneter a array").split()))  
key=int(input("Enter the element to be searched:"))  
print(linear_search(arr, key))
```

Output:



```
===== RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week2 (a) .py =====  
eneter a array2 3 4 5 64 54  
Enter the element to be searched:5  
3  
>>>
```

Explanation:

- The program defines a function `linear_search()` to search for an element in a list.
- It takes two inputs: the array (`arr`) and the element to be searched (`key`).
- The for loop checks each element of the array one by one.
- If `arr[i] == key`, the function returns the index of the element.
- If the element is not found, the function returns 0.
- The user enters the array elements and the element to be searched.
- Finally, the program calls `linear_search()` and prints the result.

Error Table:

S. No	Error Obtained	Rectification
01.	Error in input taking of array	Rectified by split.

2)Aim: To write a Python program to search for a given element in an array using the recursive Binary Search technique, by checking whether the array is sorted and sorting it if necessary.

Program:

FOR SORTED ARRAY:

```
week2(b).py - C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week2(b).py (3.14.0)
File Edit Format Run Options Window Help
def binary_search(arr, key, low, high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == key:
        return mid
    elif arr[mid] > key:
        return binary_search(arr, key, low, mid - 1)
    else:
        return binary_search(arr, key, mid + 1, high)
    return 0
arr = list(map(int, input("enter the array").split()))
key = int(input("Enter the element to be searched:"))
print(binary_search(arr, key, 0, len(arr)))
```

FOR UNSORTED ARRAY:

```
def binary_search(arr, key, low, high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == key:
        return mid
    elif arr[mid] > key:
        return binary_search(arr, key, low, mid - 1)
    else:
        return binary_search(arr, key, mid + 1, high)
    return 0
arr = list(map(int, input("enter unsorted array").split()))
key = int(input("Enter the element to be searched:"))
if arr == sorted(arr):
    print("The array is sorted")
else:
    print("The array is not sorted")
    arr.sort()
print("The sorted array is:", arr)
print(binary_search(arr, key, 0, len(arr)))
```

Output:

```
===== RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week2(b).py =====
enter the array2 3 5 6 7 8
Enter the element to be searched:5
2
~~~
```

```
>>> ===== RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/week2(c).py =====
enter unsorted array 4 3 5 1 2 6
Enter the element to be searched:5
The array is not sorted
The sorted array is: [1, 2, 3, 4, 5, 6]
4
>>>
```

Explanation:

- The program uses Binary Search to find a given element in an array.
- The binary search () function works recursively by dividing the search range into two halves.
- It calculates the middle position using $(low + high) // 2$.
- If the middle element is equal to the key, its index is returned.
- If the middle element is greater than the key, the function searches the left half.
- If the middle element is smaller than the key, it searches the right half.
- For an unsorted array, the program first checks whether the array is sorted using `array==sorted(array)`.
- If it is not sorted, “`arrsort ()`” is used to arrange the elements in ascending order.
- Finally, the Binary Search function is called to search for the required element.

Error Table:

S.No	Error Obtained	Rectification
01.	Error in if clause statement	Rectified by keeping ==

Week-3

1) Aim: To write a Python program to sort the elements of an array in ascending order using the Bubble Sort technique.

Program:

```
def bubble_sort(arr):
    n=len(arr)

    for i in range (n):
        for j in range(0,n-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]

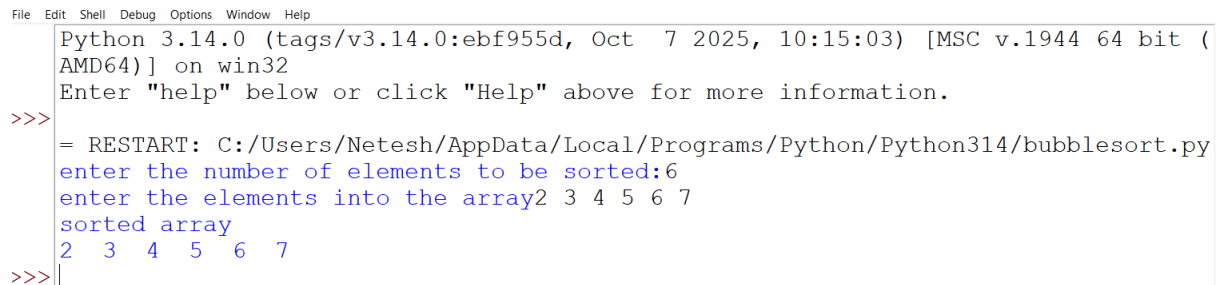
n=int(input("enter the number of elements to be sorted:"))

arr=list(map(int,input("enter the elements into the array").split()))

print("sorted array")
bubble_sort(arr)

for i in arr:
    print(i,end="  ")
|
```

Output:



```
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/bubblesort.py
enter the number of elements to be sorted:6
enter the elements into the array2 3 4 5 6 7
sorted array
2 3 4 5 6 7
>>>|
```

Explanation:

- `def bubble_sort(arr):` → Defines a function named `bubble_sort` to sort the array.
- `n = len(arr)` → Finds the number of elements in the array.
- `for i in range(n):` → Repeats the sorting process for multiple passes.
- `for j in range(0, n-i-1):` → Compares adjacent elements in the array.
- `if arr[j] > arr[j+1]:` → Checks whether the current element is greater than the next element.
- `arr[j], arr[j+1] = arr[j+1], arr[j]` → Swaps the two elements if they are in the wrong order.
- After each pass, the largest unsorted element moves to the end of the array.

- return arr → Returns the sorted array.
- numbers = [64, 34, 25, 12, 22, 11, 90] → Creates an array containing the elements to be sorted.
- print(bubble_sort(numbers)) → Calls the function and displays the sorted array.
- The final output is in ascending order:
[11, 12, 22, 25, 34, 64, 90].

Error Table:

S.no	Error	Explanation
1.	No errors	-----

2)Aim: To write a Python program to sort the elements of an array in ascending order using the Insertion Sort technique.

Program:

```
def insertion_sort(arr):
    n=len(arr)

    for i in range(1,n):
        key=arr[i]

        j=i-1

        while j>=0 and arr[j]>key:
            arr[j+1]=arr[j]

            j=j-1

        arr[j+1]=key

    return arr

arr=list(map(int,input("enter numbers to add in an array:").split()))

print(insertion_sort(arr))
```

Output:

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/insertion.py
enter numbers to add in an array:5 6 7 3 2
[2, 3, 5, 6, 7]
>>>
```

Explanation:

- Insertion Sort is a sorting technique that builds the sorted array one element at a time.
- It works similar to the way we arrange playing cards in our hand.
- The first element is considered already sorted.
- The next element is selected and called the key.
- The key is compared with the elements on its left side.
- If an element on the left is greater than the key, it is shifted one position to the right.
- This comparison and shifting continues until the correct position for the key is found.
- The key is then inserted into its correct position.
- The same process is repeated for all remaining elements.
- At the end, the complete array is sorted in ascending order.

Error Table:

S.No	Error Obtained	Explanation
1)	No errors	-----

3)Aim: To write a program to sort a given array of elements in ascending order using the Selection Sort technique.

Program:

```
def selection_sort(arr):  
    n=len(arr)  
    for i in range (n-1):  
        min_index=i  
        for j in range (i+1,n):  
            if arr[j]<arr[min_index]:  
                min_index=j  
  
        arr[i],arr[min_index]=arr[min_index], arr[i]  
  
    return arr  
arr=list(map(int,input("enter numbers to be added in an array").split()))  
print(selection_sort(arr))
```

Sapuru Netesh (netesh0316@outlook.com) is signed in

Output:

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
>>>  
= RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/selection sort.py  
enter numbers to be added in an array6 5 7 3 4 1  
[1, 3, 4, 5, 6, 7]  
>>>
```

Explanation:

- Selection Sort is a simple sorting algorithm used to arrange elements in ascending or descending order.
- It divides the array into two parts:
- Sorted part
- Unsorted part
- In each pass, it finds the smallest element from the unsorted part.
- The smallest element is then swapped with the first element of the unsorted part.
- After every pass, the sorted part becomes one element larger.
- This process continues until all elements are sorted.
- Selection Sort performs sorting in-place, so it does not require an extra array.
- The time complexity of Selection Sort is $O(n^2)$.
- It is easy to understand and implement, but it is not efficient for large datasets.

Error Table:

S.No	Errors obtained	Explanation
1.	Error in function call.	Rectified it by proper function call.
2.	Error in for loop	Rectified it by keeping n-1.

4)Aim: To write a program to sort a given array of elements in ascending order using the Merge Sort algorithm.

Program:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]
    merge_sort(left)
    merge_sort(right)
    i = j = k = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            arr[k] = left[i]
            i += 1
        else:
            arr[k] = right[j]
            j += 1
        k += 1

    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1
    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1

n = int(input("enter the number of elements to sort"))
arr = list(map(int, input("enter the elements").split()))
```

Output:

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/mergesort.py
enter the number of elements to sort5
enter the elements4 5 3 2 1
sorted array
1 2 3 4 5
>>>
```

Explanation:

- Merge Sort is a sorting algorithm based on the Divide and Conquer technique.
- It divides the array into two smaller parts repeatedly.
- This division continues until each part contains only one element.
- A single element is considered already sorted.
- The smaller parts are then merged together in the correct order.
- During merging, the elements of the two parts are compared one by one.
- The smaller element is placed first in the merged array.
- This merging process continues until the entire array is sorted.
- Merge Sort has a time complexity of $O(n \log n)$ It requires extra memory for the merging process.

Error Table:

S.No	Errors Obtained	Explanation
1.	No errors.	-----

5)Aim: To write a program to sort a given array of elements in ascending order using the Quick Sort algorithm based on the Divide and Conquer technique.

Program:

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]
    left = []
    right = []

    for x in arr[:-1]:
        if x <= pivot:
            left.append(x)
        else:
            right.append(x)

    return quick_sort(left) + [pivot] + quick_sort(right)

n = int(input("Enter number of elements: "))
arr = list(map(int, input("Enter elements: ").split()))

sorted_arr = quick_sort(arr)
print("Sorted array:", sorted_arr)
```

Output:

```
>>> = RESTART: C:/Users/Netesh/AppData/Local/Programs/Python/Python314/quicksort.py
Enter number of elements: 5
Enter elements: 7 8 5 4 3
Sorted array: [3, 4, 5, 7, 8]
>>> |
```

Explanation:

- Quick Sort is a sorting algorithm based on the Divide and Conquer technique.
- It selects one element from the array as a pivot.
- The elements are divided into two parts based on the pivot:
 - Elements smaller than the pivot are placed on the left.
 - Elements greater than the pivot are placed on the right.
- The pivot is now in its correct position in the sorted array.
- The same process is repeated for the left and right sub-arrays.
- This process continues until the sub-arrays contain one or no elements.
- Finally, all the elements are arranged in ascending order.
- Quick Sort has an average time complexity of $O(n \log n)$.
- Its worst-case time complexity is $O(n^2)$, depending on the choice of pivot.

Errors Table:

S.No	Errors obtained	Explanation
1.	No errors	-----

WEEK – 4

4a. AIM: Write a python program to implement a single linked list which would perform the below operations, by taking an input from the user.

- a. Create a linked list.
- b. Insert at beginning.
- c. Insert at end.
- d. Insert at a specific index.
- e. Delete by value.
- f. Delete 1st node.
- g. Delete last node.
- h. Count no. of nodes
- i. Display.
- j. Exit.

Program:

```
week4.py - C:/Users/ACER/Documents/week4.py (3.14.6)
File Edit Format Run Options Window Help

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    # a. Create a linked list
    def create(self):
        n = int(input("Enter the number of nodes: "))

        for i in range(n):
            data = int(input("Enter the data: "))
            self.insert_end(data)

        print("Linked list created successfully.")

    # b. Insert at beginning
    def insert_beginning(self):
        data = int(input("Enter the data: "))

        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

        print("Node inserted at beginning.")

    # c. Insert at end
    def insert_end(self, data=None):
        if data is None:
            data = int(input("Enter the data: "))

        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        temp = self.head

        while temp.next is not None:
            temp = temp.next

        temp.next = new_node

        if data is not None:
            print("Node inserted at end.")

    # d. Insert at a specific index
    def insert_at_index(self):
        index = int(input("Enter the index: "))
        data = int(input("Enter the data: "))

        if index < 0:
            print("Invalid index.")
            return

        if index == 0:
```

```
week4.py - C:/Users/ACER/Documents/week4.py (3.14.6)
File Edit Format Run Options Window Help

if index == 0:
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
    print("Node inserted at index", index)
    return

temp = self.head

for i in range(index - 1):
    if temp is None:
        print("Index out of range.")
        return
    temp = temp.next

if temp is None:
    print("Index out of range.")
    return

new_node = Node(data)
new_node.next = temp.next
temp.next = new_node

print("Node inserted at index", index)

# e. Delete by value
def delete_by_value(self):
    value = int(input("Enter the value to delete: "))

    if self.head is None:
        print("List is empty.")
```

```
week4.py - C:/Users/ACER/Documents/week4.py (3.14.6)
File Edit Format Run Options Window Help

        print("List is empty.")
        return

    if self.head.data == value:
        self.head = self.head.next
        print("Node deleted successfully.")
        return

    temp = self.head

    while temp.next is not None:
        if temp.next.data == value:
            temp.next = temp.next.next
            print("Node deleted successfully.")
            return
        temp = temp.next

    print("Value not found.")

# f. Delete first node
def delete_first(self):
    if self.head is None:
        print("List is empty.")
        return

    self.head = self.head.next
    print("First node deleted.")

# g. Delete last node
def delete_last(self):
    if self.head is None:
```

```
week4.py - C:/Users/ACER/Documents/week4.py (3.14.6)
File Edit Format Run Options Window Help

    if self.head is None:
        print("List is empty.")
        return

    if self.head.next is None:
        self.head = None
        print("Last node deleted.")
        return

    temp = self.head

    while temp.next.next is not None:
        temp = temp.next

    temp.next = None
    print("Last node deleted.")

# h. Count number of nodes
def count_nodes(self):
    count = 0
    temp = self.head

    while temp is not None:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)

# i. Display
def display(self):
    if self.head is None:
```

```
week4.py - C:/Users/ACER/Documents/week4.py (3.14.6)
File Edit Format Run Options Window Help

# Main program
linked_list = SinglyLinkedList()

while True:
    print("\n----- SINGLY LINKED LIST MENU -----")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        linked_list.create()

    elif choice == 2:
        linked_list.insert_beginning()

    elif choice == 3:
        linked_list.insert_end()

    elif choice == 4:
        linked_list.insert_at_index()

    elif choice == 5:
```

```
week4.py - C:/Users/ACER/Documents/week4.py (3.14.6)
File Edit Format Run Options Window Help

elif choice == 2:
    linked_list.insert_beginning()

elif choice == 3:
    linked_list.insert_end()

elif choice == 4:
    linked_list.insert_at_index()

elif choice == 5:
    linked_list.delete_by_value()

elif choice == 6:
    linked_list.delete_first()

elif choice == 7:
    linked_list.delete_last()

elif choice == 8:
    linked_list.count_nodes()

elif choice == 9:
    linked_list.display()

elif choice == 10:
    print("Exiting program...")
    break

else:
    print("Invalid choice. Please try again.")

Ln 1 Col 11
```

Output:

```
"IDLE Shell 3.14.6"
File Edit Shell Debug Options Window Help

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ACER/Documents/week4.py =====

----- SINGLY LINKED LIST MENU -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter the number of nodes: 3
Enter the data: 10
Enter the data: 20
Node inserted at end.
Enter the data: 30
Node inserted at end.
Linked list created successfully.

----- SINGLY LINKED LIST MENU -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value

Ln 37 Col 19
```

```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help

Node inserted at end.
Enter the data: 30
Node inserted at end.
Linked list created successfully.

----- SINGLY LINKED LIST MENU -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice: 9
Linked List: 10 -> 20 -> 30 -> None

----- SINGLY LINKED LIST MENU -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice: |
```

```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help

5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice: 4
Enter the index: 3
Enter the data: 10
Index out of range.

----- SINGLY LINKED LIST MENU -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice: 4
Enter the index: 10 20 30
Traceback (most recent call last):
  File "C:/Users/ACER/Documents/week4.py", line 194, in <module>
    linked_list.insert_at_index()
  File "C:/Users/ACER/Documents/week4.py", line 54, in insert_at_index
    index = int(input("Enter the index: "))
ValueError: invalid literal for int() with base 10: '10 20 30'
>>> |
```

EXPLANATION:

◆ Class Definitions

1. Node class

- Represents a single node in the linked list.
- Each node has:
 - data → stores the value.
 - next → pointer to the next node (initially None).

2. SLL class

- Manages the linked list operations.
- Has a head pointer (initially None) that points to the first node.

◆ Operations in SLL

1. Create Linked List

- Asks user for number of nodes.
- For each node:
 - Creates a new Node.
 - If list is empty → new node becomes head.
 - Otherwise → traverses to the end and attaches the new node.

2. Insert at Beginning

- Creates a new node.
- Points its next to current head.
- Updates head to new node.

3. Insert at End

- Creates a new node.
- If list is empty → new node becomes head.
- Otherwise → traverses to last node and attaches new node.

4. Insert at Index

- Asks for index and data.
- If index = 0 → behaves like "insert at beginning".
- Otherwise → traverses to (index-1) node and inserts new node in between.

5. Delete from Beginning

- Moves head to head.next.
- Effectively removes the first node.

6. Delete from End

- If list empty → prints "List is empty".
- If only one node → sets head = None.
- Otherwise → traverses to second-last node and sets its next = None.

7. Delete from Index

- Asks for index.
- If index = 0 → removes head.
- Otherwise → traverses to (index-1) node and skips the next node.

8. Count Nodes

- Traverses through list.
- Counts nodes until None.

9. Display

- Traverses from head to end.
- Prints each node's data in data -> data -> None format.

◆ Main Program

- Runs an infinite loop with a menu:
 - Options 1–9 → perform linked list operations.
 - Option 10 → exits program.
- Uses if-elif to call respective methods.

Error Table:

S.NO	Error Obtained	Rectification
1.	Index out of range When inserting/deleting at an index greater than the number of nodes.	Traverse carefully; if temp becomes None before reaching index, print "Index out of range" and stop.
2.	Indentation Error In the Count node part	It was rectified later, By maintaining proper indentation.

4b. AIM: Write a python program to implement a double linked list which would perform the below operations, by taking an input from the user.

- a. Create a linked list. b. Insert at beginning. c. Insert at end.
 d. Insert at a specific index. e. Delete by value. f. Delete 1st node.
 g. Delete last node. h. Count no. of nodes i. Display. J.exit.

Program:

```
week4b.py - C:/Users/ACER/Documents/week4b.py (3.14.6)
File Edit Format Run Options Window Help

class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    # a. Create a linked list
    def create(self):
        data = int(input("Enter the data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

        print("Node created successfully")

    # b. Insert at beginning
    def insert_beginning(self):
        data = int(input("Enter the data: "))
        new_node = Node(data)

        if self.head is not None:
            new_node.next = self.head
            self.head.prev = new_node

        self.head = new_node
        print("Node inserted at beginning")

    # c. Insert at end
    def insert_end(self):
```

```
week4b.py - C:/Users/ACER/Documents/week4b.py (3.14.6)
File Edit Format Run Options Window Help

    # c. Insert at end
    def insert_end(self):
        data = int(input("Enter the data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

        print("Node inserted at end")

    # d. Insert at a specific index
    def insert_index(self):
        data = int(input("Enter the data: "))
        index = int(input("Enter the index: "))

        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            new_node = Node(data)
            new_node.next = self.head

            if self.head is not None:
                self.head.prev = new_node

            self.head = new_node
            print("Node inserted successfully")
            return

        temp = self.head
        for i in range(index - 1):
            if temp is None:
                print("Invalid index")
                return
```

```
week4b.py - C:/Users/ACER/Documents/week4b.py (3.14.6)
File Edit Format Run Options Window Help

    if temp is None:
        print("Invalid index")
        return

    new_node = Node(data)
    new_node.next = temp.next
    new_node.prev = temp

    if temp.next is not None:
        temp.next.prev = new_node

    temp.next = new_node

    print("Node inserted successfully")

# e. Delete by value
def delete_value(self):
    value = int(input("Enter the value to delete: "))
    temp = self.head

    while temp is not None:
        if temp.data == value:
            if temp.prev is not None:
                temp.prev.next = temp.next
            else:
                self.head = temp.next

            if temp.next is not None:
                temp.next.prev = temp.prev

            print("Node deleted successfully")
            return

        temp = temp.next

    print("Value not found")

# f. Delete first node
def delete_first(self):
    if self.head is None:
        print("List is empty")
        return

    self.head.prev = None
    print("First node deleted")

# g. Delete last node
def delete_last(self):
    if self.head is None:
        print("List is empty")
        return

    temp = self.head

    while temp.next is not None:
        temp = temp.next

    if temp.prev is None:
        self.head = None
    else:
        temp.prev.next = None

    print("Last node deleted")

# h. Count number of nodes
def count(self):
    temp = self.head
    count = 0

    while temp is not None:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)

# i. Display
def display(self):
    if self.head is None:
        print("List is empty")
        return

    temp = self.head
```

```
week4b.py - C:/Users/ACER/Documents/week4b.py (3.14.6)
File Edit Format Run Options Window Help

    if self.head is not None:
        self.head.prev = None

    print("First node deleted")

# g. Delete last node
def delete_last(self):
    if self.head is None:
        print("List is empty")
        return

    temp = self.head

    while temp.next is not None:
        temp = temp.next

    if temp.prev is None:
        self.head = None
    else:
        temp.prev.next = None

    print("Last node deleted")

# h. Count number of nodes
def count(self):
    temp = self.head
    count = 0

    while temp is not None:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)

# i. Display
def display(self):
    if self.head is None:
        print("List is empty")
        return

    temp = self.head
```

```
week4b.py - C:/Users/ACER/Documents/week4b.py (3.14.6)
File Edit Format Run Options Window Help
temp = temp.next

    print("Number of nodes:", count)

# 1. Display
def display(self):
    if self.head is None:
        print("List is empty")
        return

    temp = self.head

    print("Doubly Linked List:")

    while temp is not None:
        print(temp.data, end=" <-> ")
        temp = temp.next

    print("None")

# Main program
dll = DoublyLinkedList()

while True:
    print("\n--- DOUBLY LINKED LIST MENU ---")
    print("1. Create a linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
```

```
week4b.py - C:/Users/ACER/Documents/week4b.py (3.14.6)
File Edit Format Run Options Window Help
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
        dll.insert_beginning()

    elif choice == 3:
        dll.insert_end()

    elif choice == 4:
        dll.insert_index()

    elif choice == 5:
        dll.delete_value()

    elif choice == 6:
        dll.delete_first()

    elif choice == 7:
        dll.delete_last()

    elif choice == 8:
        dll.count()

    elif choice == 9:
        dll.display()

    elif choice == 10:
        print("Exiting program...")
        break

    else:
        print("Invalid choice")
```

Output:

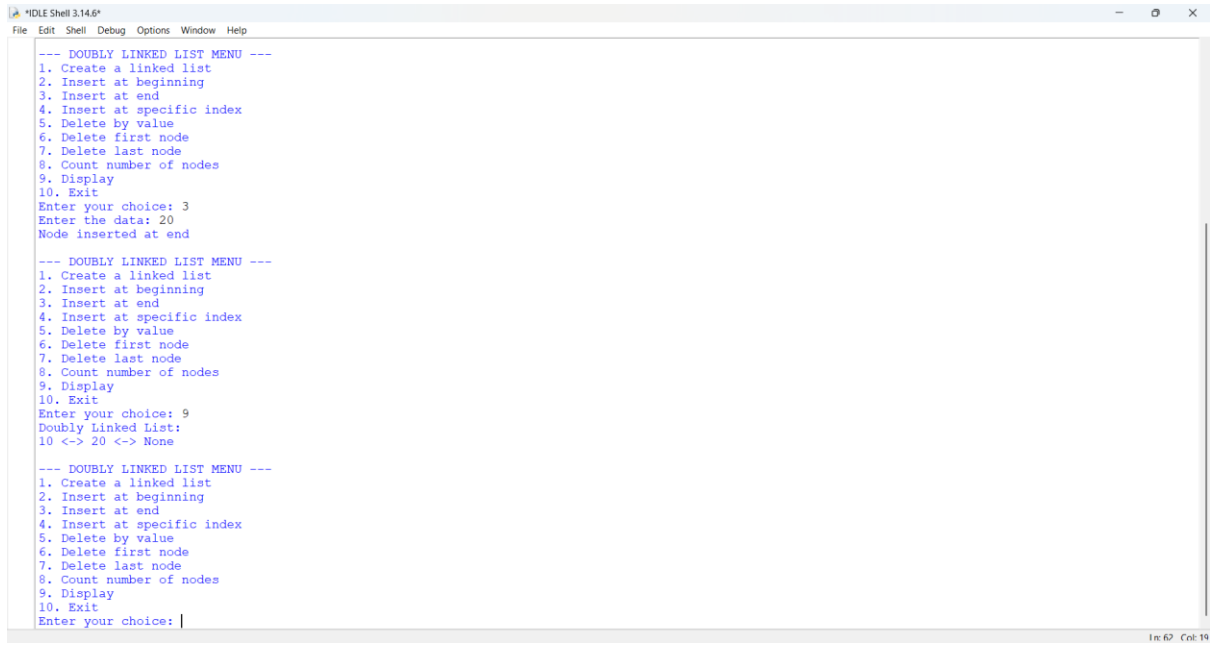
```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ACER/Documents/week4b.py =====

--- DOUBLY LINKED LIST MENU ---
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter the data: 10 20 30
Traceback (most recent call last):
  File "C:/Users/ACER/Documents/week4b.py", line 202, in <module>
    dll.create()
  File "C:/Users/ACER/Documents/week4b.py", line 15, in create
    data = int(input("Enter the data: "))
ValueError: invalid literal for int() with base 10: '10 20 30'
>>>
```

```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ACER/Documents/week4.py =====

----- SINGLY LINKED LIST MENU -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice: 4
Enter the index: 2
Enter the data: 20
Index out of range.

----- SINGLY LINKED LIST MENU -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice: |
```



```
--- DOUBLY LINKED LIST MENU ---
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter the data: 20
Node inserted at end

--- DOUBLY LINKED LIST MENU ---
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice: 9
Doubly Linked List:
10 <-> 20 <-> None

--- DOUBLY LINKED LIST MENU ---
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Exit
Enter your choice: |
```

EXPLANATION:

1. Node Class

- Each node stores:
 - data: the actual value.
 - prev: pointer to the previous node.
 - next: pointer to the next node.
- This structure allows traversal in both directions (forward and backward).

2. Double Linked List Class

This class manages the linked list. It starts with `self.head = None` (empty list). It provides several operations:

a. Create Linked List

- User specifies how many nodes to create.
- Each new node is linked at the end.
- prev pointer connects the new node to the previous one.

b. Insert at Beginning

- A new node is created.
- Its next points to the current head.
- The old head's prev points back to the new node.
- The new node becomes the head.

c. Insert at End

- Traverse to the last node.
- Attach the new node at the end.
- Update prev of the new node to point to the last node.

d. Insert at Specific Index

- Traverse until the given index.
- Insert the new node between two nodes.
- Adjust both next and prev pointers to maintain the chain.

e. Delete by Value

- Search for the node with the given value.
- If found:
 - If it's the head, move head to the next node.
 - Otherwise, adjust prev.next and next.prev to bypass the node.
- This removes the node from the chain.

f. Delete First Node

- Move head to the second node.
- Set its prev to None.

g. Delete Last Node

- Traverse to the last node.
- Remove it by setting its previous node's next to None.

h. Count Nodes

- Traverse the list, incrementing a counter.
- Prints the total number of nodes.

i. Display

- Traverse from head to end.
- Print each node's data in the format: data <-> data <-> ... None

3. Main Program (Menu-Driven)

- Runs an infinite loop with options:
 1. Create Linked List
 2. Insert at Beginning

3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Nodes
9. Display
- 10.Exit

Based on user choice, the corresponding method is executed

Error table:

S.NO	Error Obtained	Rectification
1.	Invalid index entered (negative or too large).	Add a check: if index < 0 → print "Invalid index"; if traversal reaches None before index → print "Index out of range".
2.	Deleting from an empty list.	Before deletion, check if self.head is None: → print "Doubly Linked List is empty."
3.	Value not found during deletion by value.	Traverse list; if node not found → print "value not found in the Linked List."
4.	Insertion at index when list is empty but index > 0.	Handle separately: if self.head is None and index! = 0 → print "Index out of range."

4c. AIM: Write a python program to implement a circular linked list which would perform the below operations, by taking an input from the user.

- a. Create a linked list.
- b. Insert at beginning.
- c. Insert at end.
- d. Insert at a specific index.
- e. Delete by value.
- f. Delete 1st node.
- g. Delete last node.
- h. Count no. of nodes
- i. Display.
- j. Display head and tail.
- k. Print data from tail to head.
- l. Exit.

Program:

```
untitled (3.14.6)
File Edit Format Run Options Window Help

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    # a. Create a linked list
    def create(self):
        data = int(input("Enter the data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
        else:
            new_node.next = self.head
            self.tail.next = new_node
            self.tail = new_node

        print("Node created successfully")

    # b. Insert at beginning
    def insert_beginning(self):
        data = int(input("Enter the data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
        else:
            new_node.next = self.head
            self.head = new_node
            self.tail.next = self.head

        print("Node inserted at beginning")
```

```
week4.py - C:/Users/ACER/Documents/week4.py (3.14.6)
File Edit Format Run Options Window Help

# c. Insert at end
def insert_end(self, data=None):
    if data is None:
        data = int(input("Enter the data: "))

    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        return

    temp = self.head

    while temp.next is not None:
        temp = temp.next

    temp.next = new_node

    if data is not None:
        print("Node inserted at end.")

# d. Insert at a specific index
def insert_at_index(self):
    index = int(input("Enter the index: "))
    data = int(input("Enter the data: "))

    if index < 0:
        print("Invalid index.")
        return

    if index == 0:
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        print("Node inserted at index", index)
        return

    temp = self.head

    for i in range(index - 1):
        if temp is None:
            print("Index out of range.")
            return
```

```
week4c.py - C:/Users/ACER/Documents/week4c.py (3.14.6)
File Edit Format Run Options Window Help

# e. Delete by value
def delete_value(self):
    value = int(input("Enter the value to delete: "))

    if self.head is None:
        print("List is empty")
        return

    current = self.head
    previous = self.tail

    while True:
        if current.data == value:
            if self.head == self.tail:
                self.head = None
                self.tail = None
            else:
                previous.next = current.next
                if current == self.head:
                    self.head = current.next
                if current == self.tail:
                    self.tail = previous
                self.tail.next = self.head
            print("Node deleted successfully")
            return

        previous = current
        current = current.next

    if current == self.head:
        break

    print("Value not found")

# f. Delete first node
def delete_first(self):
```

Ln: 2 Col: 0

```
week4c.py - C:/Users/ACER/Documents/week4c.py (3.14.6)
File Edit Format Run Options Window Help

# f. Delete first node
def delete_first(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head == self.tail:
        self.head = None
        self.tail = None
    else:
        self.head = self.head.next
        self.tail.next = self.head

    print("First node deleted")

# g. Delete last node
def delete_last(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head == self.tail:
        self.head = None
        self.tail = None
    else:
        temp = self.head
        while temp.next != self.tail:
            temp = temp.next

        temp.next = self.head
        self.tail = temp

    print("Last node deleted")

# h. Count number of nodes
def count(self):
    if self.head is None:
        print("Number of nodes: 0")
        return

    count = 0
```

Ln: 2 Col: 0

```
week4c.py - C:/Users/ACER/Documents/week4c.py (3.14.6)
File Edit Format Run Options Window Help

# h. Count number of nodes
def count(self):
    if self.head is None:
        print("Number of nodes: 0")
        return

    count = 0
    temp = self.head

    while True:
        count += 1
        temp = temp.next

        if temp == self.head:
            break

    print("Number of nodes:", count)

# i. Display
def display(self):
    if self.head is None:
        print("List is empty")
        return

    temp = self.head
    print("Circular Linked List:")

    while True:
        print(temp.data, end=" -> ")
        temp = temp.next

        if temp == self.head:
            break

    print("HEAD")

# j. Display head and tail
def display_head_tail(self):
    if self.head is None:
        print("List is empty")
        return

    print("Tail:", self.tail.data)

# k. Print data from tail to head
def print_tail_to_head(self):
    if self.head is None:
        print("List is empty")
        return

    values = []
    temp = self.head

    while True:
        values.append(temp.data)
        temp = temp.next

        if temp == self.head:
            break

    print("Data from tail to head:")

    for data in reversed(values):
        print(data, end=" -> ")

    print("HEAD")

# Main program
c11 = CircularLinkedList()

while True:
    print("\n--- CIRCULAR LINKED LIST MENU ---")
    print("1. Create a linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display")
    print("10. Display head and tail")
    print("11. Print data from tail to head")
    print("12. Exit")
```

```
week4c.py - C:/Users/ACER/Documents/week4c.py (3.14.6)
File Edit Format Run Options Window Help

choice = int(input("Enter your choice: "))

if choice == 1:
    cll.create()
elif choice == 2:
    cll.insert_beginning()
elif choice == 3:
    cll.insert_end()
elif choice == 4:
    cll.insert_index()
elif choice == 5:
    cll.delete_value()
elif choice == 6:
    cll.delete_first()
elif choice == 7:
    cll.delete_last()
elif choice == 8:
    cll.count()
elif choice == 9:
    cll.display()
elif choice == 10:
    cll.display_head_tail()
elif choice == 11:
    cll.print_tail_to_head()
elif choice == 12:
    print("Exiting program...")
    break
else:
    print("Invalid choice")
```

Ln: 2 Col: 0

Output:

```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/ACER/Documents/week4c.py =====

--- CIRCULAR LINKED LIST MENU ---
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Display head and tail
11. Print data from tail to head
12. Exit
Enter your choice: 1
Enter the data: 10 20
Traceback (most recent call last):
  File "C:/Users/ACER/Documents/week4c.py", line 273, in <module>
    cll.create()
  File "C:/Users/ACER/Documents/week4c.py", line 15, in create
    data = int(input("Enter the data: "))
ValueError: invalid literal for int() with base 10: '10 20'

>>>
```

Ln: 27 Col: 0

```
*IDLE Shell 3.14.6*
File Edit Shell Debug Options Window Help

5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Display head and tail
11. Print data from tail to head
12. Exit
Enter your choice: 3
Enter the data: 20
Node inserted at end

--- CIRCULAR LINKED LIST MENU ---
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Display head and tail
11. Print data from tail to head
12. Exit
Enter your choice: 9
Circular Linked List:
10 -> 20 -> HEAD

--- CIRCULAR LINKED LIST MENU ---
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display
10. Display head and tail
11. Print data from tail to head
12. Exit
Enter your choice: |
```

EXPLANATION:

1. Node Class

- Each node stores:
 - data: the value.
 - next: pointer to the next node.
- In a circular linked list, the last node's next points back to the head, forming a circle.

2. Circular Linked List Class

This class manages the linked list. It starts with `self.head = None` (empty list). It provides several operations:

a. Create Linked List

- User specifies number of nodes.
- Each new node is linked at the end.
- The last node's next points back to the head, ensuring circularity.

b. Insert at Beginning

- A new node is created.
- It becomes the new head.
- The last node's next is updated to point to this new head.

c. Insert at End

- Traverse to the last node.

- Attach the new node after it.
- New node's next points back to head.

d. Insert at Specific Index

- If index = 0 → same as inserting at beginning.
- Otherwise, traverse to the given position.
- Insert the new node between two nodes.
- Maintain circularity by adjusting next pointers.

e. Delete by Value

- Search for the node with the given value.
- If it's the head:
 - ✚ If only one node exists → set head to None.
 - Otherwise, adjust last node's next to point to new head.
- If it's in the middle → bypass the node by linking previous node's next to the next node.

f. Delete First Node

- If only one node → set head to None.
- Otherwise, move head to the second node and update last node's next.

g. Delete Last Node

- Traverse to the second-last node.
- Make its next point to head.
- Removes the last node.

h. Count Nodes

- Traverse until you return to head.
- Count nodes during traversal.

i. Display

- Print nodes starting from head.
- Stop when traversal returns to head.
- Shows circularity by ending with "HEAD".

j. Display Head and Tail

- Head = first node.

- Tail = last node (whose next points to head).
- Prints both values.

k. Print Data from Tail to Head

- Find tail.
- Traverse backward by repeatedly searching for the node whose next points to the current node.
- Prints values in reverse order until reaching head.

3. Main Program (Menu-Driven)

- Runs an infinite loop with give options.
- Based on user choice, the corresponding method is executed.

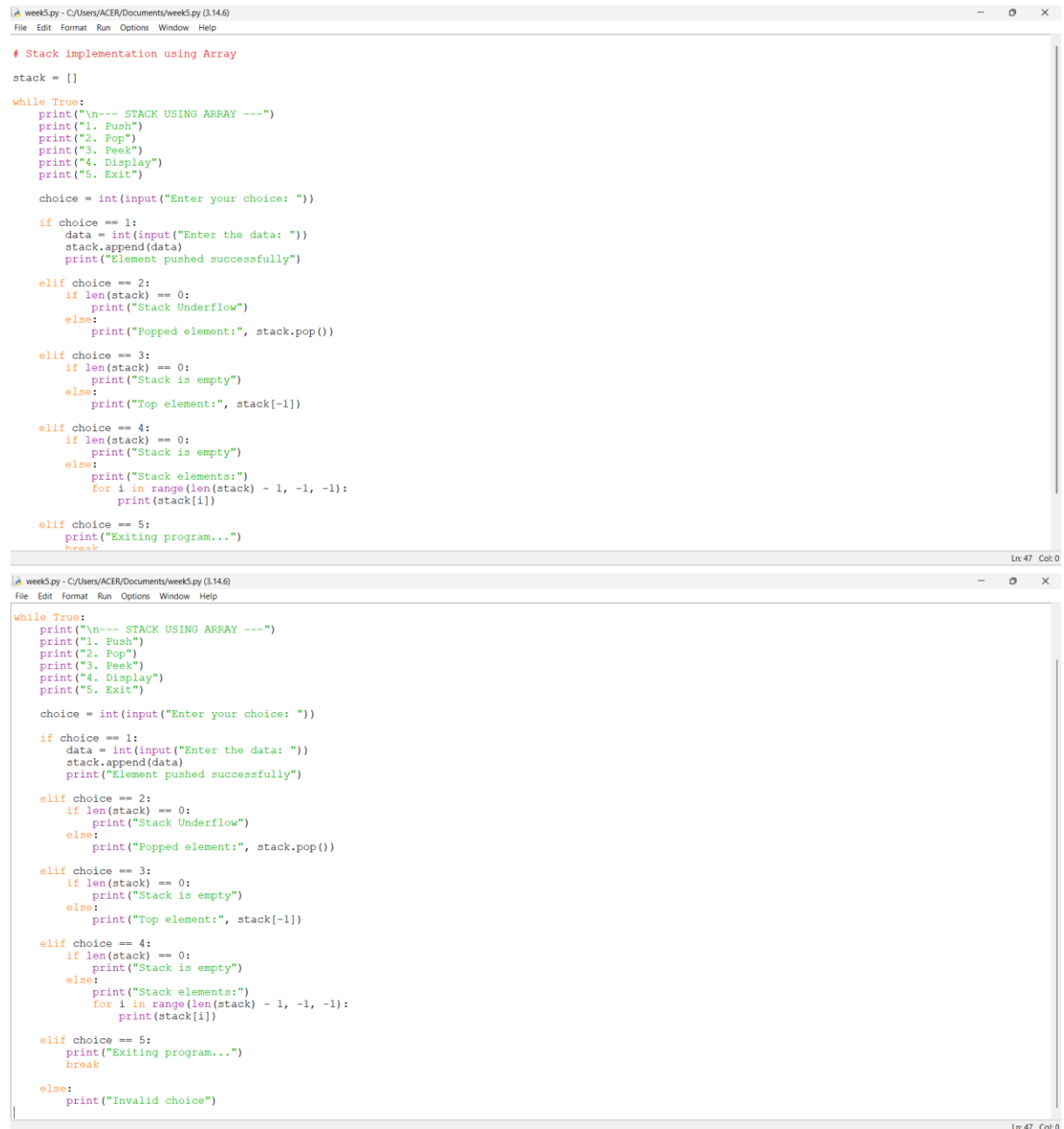
ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Display on empty list.	Add condition: if self.head is None: → print "Circular Linked List is empty."
2.	Reverse traversal (Tail to Head) inefficient.	Current implementation repeatedly searches for predecessor. Rectify by using a doubly circular linked list if reverse traversal is frequent.

WEEK – 5

5. AIM: Write a python program to implement a stack using an Array, Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.

Program:



```
# Stack implementation using Array
stack = []

while True:
    print("\n--- STACK USING ARRAY ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the data: "))
        stack.append(data)
        print("Element pushed successfully")

    elif choice == 2:
        if len(stack) == 0:
            print("Stack Underflow")
        else:
            print("Popped element:", stack.pop())

    elif choice == 3:
        if len(stack) == 0:
            print("Stack is empty")
        else:
            print("Top element:", stack[-1])

    elif choice == 4:
        if len(stack) == 0:
            print("Stack is empty")
        else:
            print("Stack elements:")
            for i in range(len(stack) - 1, -1, -1):
                print(stack[i])

    elif choice == 5:
        print("Exiting program...")
        break

    else:
        print("Invalid choice")
```

Output:

```
*IDLE Shell 3.14.6*
File Edit Shell Debug Options Window Help

2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the data: 10
Element pushed successfully

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 10

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 10

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements:
10

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
```

Stack Using Linked List:

```
week5apy - C:/Users/ACER/Documents/week5apy (3.14.6)
File Edit Format Run Options Window Help

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    # Push operation
    def push(self):
        data = int(input("Enter the data: "))

        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node

        print("Element pushed successfully")

    # Pop operation
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Popped element:", self.top.data)
            self.top = self.top.next

    # Peek operation
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    # Display operation
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
```

```
week5a.py - C:/Users/ACER/Documents/week5a.py (3.14.6)
File Edit Format Run Options Window Help
    print("Stack is empty")
else:
    temp = self.top
    print("Stack elements:")
    while temp is not None:
        print(temp.data)
        temp = temp.next

# Main program
s = Stack()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        s.push()

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Exiting program...")
        break

    else:
        print("Invalid choice")
```

Output:

```
*IDLE Shell 3.14.6*
File Edit Shell Debug Options Window Help
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the data: 10
Element pushed successfully

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 10

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 20
Invalid choice

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Stack is empty

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: |
```

EXPLANATION:

Stack Using Array

Structure:

- Uses a Python list (self.stack) to store elements.
- self.top keeps track of the index of the last pushed element.
- self.size defines the maximum capacity.

Operations:

1. Push
 - Check if $\text{top} == \text{size} - 1 \rightarrow \text{Overflow}$.
 - Otherwise, increment top and insert the element at that position.
2. Pop
 - Check if $\text{top} == -1 \rightarrow \text{Underflow}$ (empty stack).
 - Otherwise, remove the element at top, decrement top.
3. Peek
 - Shows the element at top without removing it.
4. Display
 - Prints elements from top down to 0 (top to bottom order).

✿ Stack Using Linked List

Structure:

- Each element is a Node with:
 - data (value)
 - next (pointer to the next node).
- self.top points to the most recently pushed node.

Operations:

1. Push
 - Create a new node.
 - Link its next to the current top.
 - Update top to this new node.
2. Pop
 - If top is None $\rightarrow$ Underflow.
 - Otherwise, remove the node at top and update top to top.next.
3. Peek
 - Shows the data at top.
4. Display
 - Traverse from top downwards, printing each node's data.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	def _init_(self, size):	Python expects two underscores before and after init.
2.	Display on empty stack.	Add condition: if self.top is None: → print "Stack is empty".
3.	Syntax error	Later it was rectified in the for loop of display section.

